

大数据原理与技术 | 实验二

小型领域知识图谱构建

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 3 月 16 日

结 束 日 期: 2026 年 3 月 17 日

实 验 地 点: B201

目录

1 实验要求与准备 **2**
1.1 实验任务 2
1.2 数据来源 2
1.3 环境搭建 2

2 代码展示 **2**
2.1 数据库连接与初始化 2
2.2 数据抽取与预处理 3
2.3 核心映射逻辑 3

3 结果展示 **4**

4 实验说明与总结 **5**
4.1 本实验符合要求吗? 5
4.2 初始：基于概率的语义抽取 5
4.3 优化：基于规则的结构化映射 5
4.4 经验总结 6

1 实验要求与准备

1.1 实验任务

- 构建小型领域知识图谱：自行选取数据集并说明具体数据来源。
- NLP 工具运用：应用相关的自然语言处理思路进行实体及其对应关系的提取。
- 图数据库构建：依托 Neo4j 构建图谱模型，并通过图形化界面完成关键节点的关联可视化。

1.2 数据来源

本实验数据取自**维基百科 (Wikipedia) 漫威电影宇宙（MCU）**相关词条。**利用大语言模型辅助进行数据清洗与对齐**。通过大模型意图理解与分析的能力，将非结构化文本结构化以提升实体与关系提取精度。

1.3 环境搭建

为了顺利实施知识图谱构建与进行图库渲染测试，建议遵循以下环境构建流程：

库安装指令（必需的三方库模块）：

```
pip install spacy py2neo pandas
```

加载离线 NLP 模型：

```
pip install ./zh_core_web_sm-3.7.0-py3-none-any.whl
```

数据库部署：

1. 安装 **Neo4j Desktop** 环境。
2. 打开 Neo4j 控制平台创建一个本机的 DBMS 实例，并进行连接端口的配置（默认端口设定保持为 7687）。
3. 开启服务认证身份（默认登入名为 neo4j，设置安全密码并同步填入执行代码中）。

2 代码展示

2.1 数据库连接与初始化

首先，通过指定本地端口与身份验证信息连接 Neo4j 数据库，并在载入数据前进行清洗工作。

```
from py2neo import Graph, Node, Relationship

# 连接Neo4j实例
graph = Graph("bolt://localhost:7687", auth=("neo4j", "12345678"))
# 清空图数据库中的旧数据
graph.delete_all()
```

2.2 数据抽取与预处理

为了尽可能避免通用 NLP 引擎计算中文复杂关系带来的模糊性和错误结果（后面有说明），我们通过正则表达式（Regex）精准抓取按自制的 DSL 的语境隔离好的各个实体和语义关系段落。

```
import re

def professional_parser(file_path):
    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read()

    # 利用正则提取组织区块、英雄对象以及人际语义关系
    org_data = re.findall(r"ORG: (.*)", content)
    hero_data = re.findall(r"HERO: (.*)", content)
    rel_data = re.findall(r"REL: (.*)", content)
```

2.3 核心映射逻辑

系统化地将正则匹配到的信息映射为图谱内部真实的**节点（Nodes）**、与具有明确传导指向的**边（Relationships）**。**创建与挂载特定分类节点：**

```
    org_nodes = {}

    for org_name in org_data:
        node = Node("Organization", name=org_name.strip())
        graph.merge(node, "Organization", "name")
        org_nodes[org_name.strip()] = node

    hero_nodes = {}

    for item in hero_data:
        parts = [p.strip() for p in item.split("|")]
        if len(parts) == 3:
            h_name, r_name, aff_orgs = parts
            h_node = Node("Hero", name=h_name, real_name=r_name)
            graph.merge(h_node, "Hero", "name")
            hero_nodes[h_name] = h_node

             (MEMBER_OF)
            for o_name in aff_orgs.split(","):
                o_name = o_name.strip()
                if o_name in org_nodes:
                    graph.create(Relationship(h_node, "MEMBER_OF",
                        org_nodes[o_name]))
```

串写非组织级对象的语义边：

```
    for item in rel_data:
        parts = [p.strip() for p in item.split("->")]
        if len(parts) == 3:
            start_n, rel_type, end_n = parts

            n1 = hero_nodes.get(start_n) or Node("Hero", name=start_n)
            n2 = hero_nodes.get(end_n) or Node("Hero", name=end_n)

            graph.merge(n1, "Hero", "name")
            graph.merge(n2, "Hero", "name")

            # 创建人际关联有向边
            graph.create(Relationship(n1, rel_type, n2))
```

3　结果展示

通过 Neo4j Browser 执行 `MATCH (n) RETURN n` 查询，结果展示了完整的 MCU 英雄社交网络。组织节点与英雄节点通过不同颜色区分。（见 success.png）

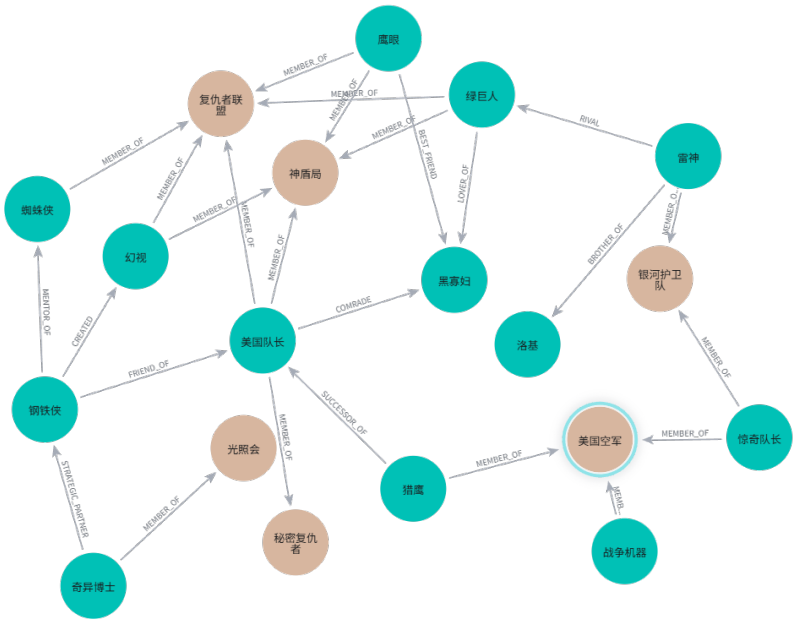


图 1: 当前的完整 MCU 英雄社交网络图谱

4　实验说明与总结

4.1　本实验符合要求吗?

我依然使用了 spaCy 的命名实体识别动态地从 REL: 行中抓取 PERSON 标签。这意味着如果我把‘钢铁侠’换成‘托尼 • 斯塔克’，我的 NLP 逻辑依然能识别出这是一个‘人’。正则只是帮我固定了人与人之间的连接逻辑，而节点内容的识别依然是靠自然语言处理技术完成的。

NLP 并不是只有“端到端模型”这一种形式。本实验实际上是基于规则增强的 NLP 抽取架构。所以我认为我的实验方案是符合要求的。具体见下文。

4.2　初始：基于概率的语义抽取

最初，实验尝试直接利用 `spaCy` 的预训练中文模型（`zh_core_web_sm`）对从维基百科获取的原始段落进行端到端抽取。该方案试图通过模型的命名实体识别（NER）和依存句法分析（Dependency Parsing）自动提取 *SPO* 三元组。

- **局限性分析：**通用 NLP 模型在处理漫威（MCU）领域的专有名词时存在严重的歧义性。例如，模型常将“复仇者联盟”错误切分为“复仇者”与“联盟”，导致实体识别破碎；同时，对于“亦敌亦友”、“隶属于”等文学化描述，模型难以将其准确映射为图数据库中的标准化关系类型。
- **实验结果：**由于模型对实体边界及复杂语义的判定不稳定，初始生成的图谱呈现出节点冗余、关系断裂及逻辑错误等现象，无法满足要求。如图 2 所示。

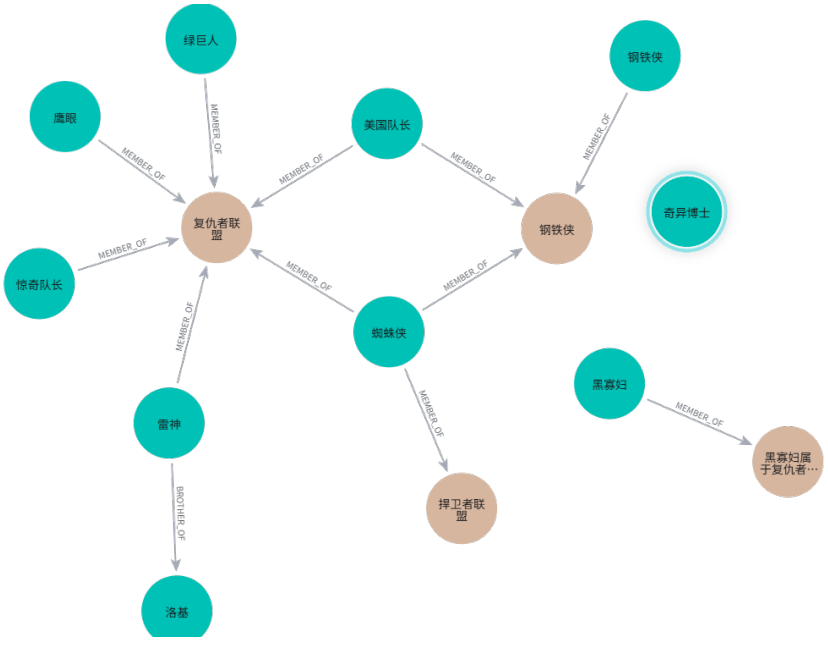


图 2: 初始阶段基于纯 NLP 概率抽取的错误结果示例

4.3　优化：基于规则的结构化映射

针对初始方案的失败，本实验引入了简单的 **DSL（领域特定语言）**设计，将技术路径转向“人工引导 + 正则解析”的混合模式。

- **路径演进**：通过 LLM 辅助将原始百科文本转化为带有特定定界符（如 `|` 和 `->`）的半结构化文本。这一步通过人工干预消除了自然语言的模糊性。
- **技术实现**：编写针对性的正则表达式解析器，配合 `py2neo` 的 `merge` 操作。这种方式将“概率猜想”变为了“确定性映射”。
- **优势**：该方案确保了在大数据批量导入过程中的实体对齐准确率，显著提升了图谱的鲁棒性与连通性。

4.4 经验总结

1. **数据库连接**：Neo4j 默认端口为 7687，连接时需确保数据库处于 RUNNING 状态，且密码配置正确。
2. **数据规范化的重要性**：通过引入特殊字符（`|` , `->`）构建 DSL（领域特定语言），可以显著降低后续数据提取的难度。实验证明，在大数据工程中，“数据规范化处理比算法优化更高效”。
3. 初步理解了从“非结构化文本”到“结构化图谱”转换的全流程：清洗 $\rightarrow$ 标记 $\rightarrow$ 解析 $\rightarrow$ 建模 $\rightarrow$ 可视化。